# AgenticAI

- Collaboration,
- Autonomy and Distribution,
- Problem-Solving

GROKKERS
AI FOR EVERYONE

# Pre-reqs

**Python Programming**

Classes, functions, decorators, file I/O

Familiarity with libraries: requests, asyncio, multiprocessing, json, logging

**Machine Learning Basics**

Supervised vs. unsupervised learning

Key models: logistic regression, decision trees, basic neural networks

Metrics: accuracy, precision, recall, ROC-AUC

**Deep Learning Foundations**

CNNs, RNNs, Transformers

Frameworks: PyTorch or TensorFlow (preferably PyTorch for agentic systems)

Loss functions, optimizers, training loops

**Prompt Engineering and GenAI Concepts**

Role of system prompts, few-shot examples

Safety and alignment basics (guardrails, hallucination handling)

**APIs and Integration**

REST APIs, JSON, token auth

Calling and hosting APIs via FastAPI or Flask

**Large Language Models**

Langchain, LlamaIndex, Haystack RAG applications

# OpenAI — Agentic AI

- **Definition/Concept**:
  Agentic AI refers to AI systems designed to **act autonomously to achieve goals** by perceiving their environment, reasoning, planning, and interacting with tools or other agents.
  OpenAI highlights **multi-step decision making, tool use, and iterative reasoning** as central to agentic AI.
- **Key points:**
  - Autonomous or semi-autonomous goal pursuit
  - Use of external tools and APIs
  - Ability to plan and execute sequences of actions
  - Supports multi-agent collaboration and handoffs
  - Emphasis on safety, interpretability, and alignment
- **Source:**
  OpenAI's blog posts on GPT agents, documentation on plugins and agent frameworks, research papers on autonomous agents.

# Hugging Face — Agentic AI

- **Definition/Concept**:
  Hugging Face describes agentic AI as **language models or AI systems equipped with agency through tool use and environment interaction**, often through frameworks like LangChain or AutoGPT.
  Agentic AI emphasizes **LLMs acting as orchestrators or planners**, combining **reasoning with tool calls**.
- **Key points:**
  - LLMs extended to act beyond just text generation
  - Integration with external APIs, databases, or scripts
  - Chains of reasoning and decision making (e.g., ReAct, Tree-of-Thought)
  - Framework-centric: enabling developers to build agents easily
  - Focus on developer experience and ecosystem support
- **Source:**
  Hugging Face blog posts, documentation on agent frameworks, and community examples.

# Anthropic — Agentic AI

- **Definition/Concept**:
  Anthropic frames agentic AI around **“constitutional AI” principles** that create agents capable of **safe, interpretable, and autonomous behavior**.
  Agentic AI here focuses on **aligned agents with constrained autonomy**, using **iterative reflection and feedback** to ensure safe actions.
- **Key points:**
  - Safety-first autonomous agents
  - Emphasis on interpretability and controllability
  - Agents that can self-reflect and learn from constitutional feedback
  - Use in complex environments requiring long-term planning
  - Research-driven focus on alignment and reliability
- **Source:**
  Anthropic research papers, blogs on constitutional AI, and their approach to agentic systems.

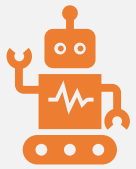
# What is agent?

**agents** are autonomous entities capable of perceiving their environment, making decisions, and acting to achieve specific goals.

# Key characteristics of agents

- Autonomy
  - Agents operate without direct human intervention, having control over their actions.
- Perception
  - They can sense and interpret information from their surroundings.
- Action
  - They can perform actions to influence their environment.
- Goal-Oriented
  - Their behavior is directed towards achieving specific objectives.
- Reactivity
  - They can respond to changes in their environment in a timely manner.

# Historical Perspective of Agents



concept of agents has roots in multiple disciplines, including artificial intelligence (AI), computer science, robotics, economics, and philosophy.

The idea of autonomous entities interacting to solve problems emerged or evolved as computing power and theoretical frameworks advanced.

# Pre-Computer Era (Philosophical Roots, Pre-20th Century)

The notion of agents can be traced to philosophical ideas about autonomy and decision-making. Aristotle's concept of *automata* (self-moving entities) and Leibniz's ideas about rational actors laid early groundwork.

In economics, Adam Smith's "invisible hand" (1776) implied individuals acting as autonomous agents in markets, producing collective outcomes without centralized control.

# 1950s–1960s: Origins in Classical AI

01

Agents were largely **symbolic**, **rule-based systems**.

02

Early examples: chess programs, theorem provers.

03

Focused on **logical reasoning**, not interaction with environments

# Key Concepts of 1970s–80s Intelligent Agents

**Perception-Action Cycle**

Agents began to **sense** their environment (perception), **process** it, and **act** accordingly — forming a feedback loop.

**Example:** A simple robot with light sensors turns toward a light source.

- **Perceives**: Light intensity from left/right.
- **Acts**: Rotates towards stronger light.

...

**2019**

**Autonomous Vehicle Fleets**

- Companies like Waymo and Tesla deploy fleets of autonomous vehicles, operating as hybrid MAS with centralized policies and decentralized navigation.

- Impact: Brings agentic systems to real-world transportation.

**2020**

**Large Language Model Agents**

- LLMs like GPT-3 enable virtual agents (e.g., chatbots, assistants) with natural language understanding, acting as deliberative agents in human-agent interactions.

- Impact: Expands agent applications to conversational and knowledge-based tasks.

# AI: The Evolution of Intelligent Systems

| | |
|---|---|
| **Rule-Based AI** (Expert Systems) | Uses pre-defined logic, IF-THEN rules, and deterministic decision-making. (e.g., early chatbots, fraud detection with business rules). |
| **Statistical Methods** | Relies on probability distributions and hypothesis testing for decision-making. (e.g., Naïve Bayes classifiers, linear regression). |
| **Machine Learning (ML)** | Models learn from data rather than being explicitly programmed. (e.g., decision trees, support vector machines). |
| **Deep Learning (DL)** | Uses neural networks to model complex patterns and relationships. (e.g., CNNs for image processing, Transformers for NLP). |
| **Reinforcement Learning (RL)** | Agents learn optimal policies by interacting with an environment and maximizing long-term rewards. (e.g., AlphaGo, self-driving cars). |

# What is Rule-Based AI?

- Rule-Based AI (also called **Expert Systems**) operates using **predefined IF-THEN rules** to make decisions.
- These systems do **not learn from data** but instead rely on **explicitly programmed logic** to handle specific scenarios.
- 🛠 **How It Works:**
  1. The system **checks conditions** based on input.
  2. If a rule is satisfied, it triggers an **action or response**.
  3. No learning or adaptation—**only predefined logic is followed**.

# Simple Example: Spam Email Filter

## 📩 Rules:

- IF an email contains **"Congratulations! You won"**, THEN mark as **Spam**.
- IF an email is from **trusted@company.com**, THEN mark as **Not Spam**.
- IF an email contains **"urgent bank details"**, THEN move to **Fraud Alert Folder**.

## 🔍 Limitations:

- Cannot handle **new spam patterns** unless manually updated.
- No ability to **learn** from new email trends.

# 📞 Customer Support Chatbot (Rule-Based Decision Tree)

💬 **User:** "I lost my debit card."

🤖 **Bot:** "Would you like to block your card? (Yes/No)"

💬 **User:** "Yes"

🤖 **Bot:** "Please provide your account number."

## 💡 How It Works:

- The chatbot follows **decision-tree logic**, where each response depends on predefined rules.

## 🔍 Limitations:

- Cannot handle **unexpected questions** (e.g., "Can I withdraw money without my card?").
- Struggles with **natural language variations** (e.g., "My card is missing" vs. "I lost my card").

# 🏦 Fraud Detection in Banking

## Example Rules

- IF a transaction is **above $10,000** AND outside the user's **home country**, THEN **flag for review**.
- IF a customer **logs in from multiple locations within 5 minutes**, THEN **lock account**.

## 🔍 Limitations:

- False positives: A user might **legitimately** travel and make a large purchase.
- Cannot adapt to **new fraud tactics** without updating rules.

# Advantages & Disadvantages of Rule-Based AI

## ✅ Advantages:

- **Transparent & Explainable** (easy to understand decision logic).
- **Reliable for well-defined tasks** (e.g., ATM PIN verification).
- **Fast execution** (no training required).

## ❌ Disadvantages:

- **Rigid & non-adaptive** (fails with new patterns).
- **High maintenance** (rules need constant updating).
- **Limited scalability** (becomes too complex as rules increase).

# Machine Learning as a Solution

✅ Unlike rule-based AI, **ML models learn patterns from data** rather than relying on manually defined rules.

✅ **Adapt to New Scenarios** – ML models can generalize from past data and handle **new, unseen cases**.

✅ **Handle Complex Decision-Making** – ML captures **patterns** that are too complicated for rule-based logic.

✅ **Scale Efficiently** – Models improve over time without requiring **manual rule updates**.

# From Rules to Supervised Learning

## Example: Email Spam Detection

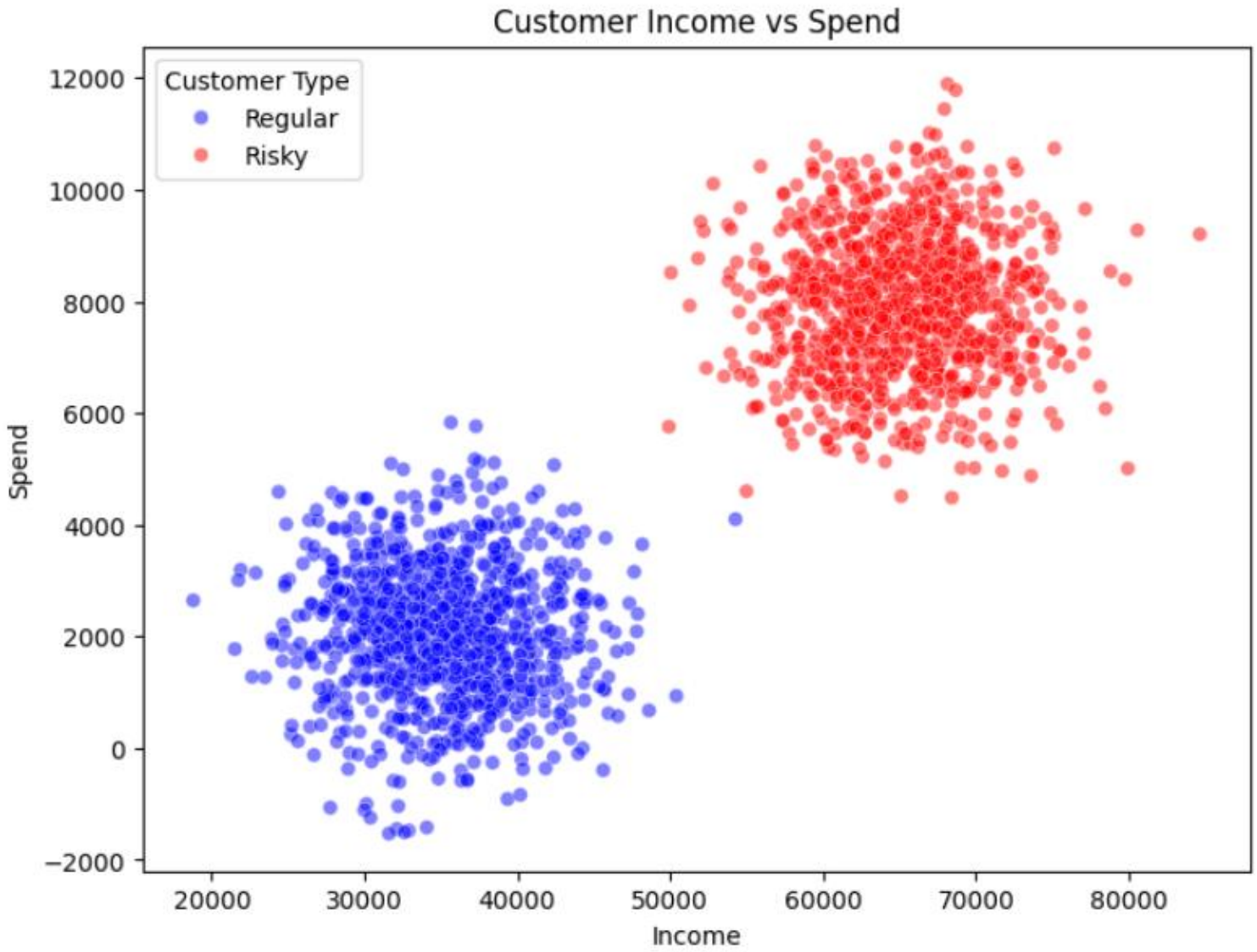
### 🔴 Rule-Based Approach:

- IF email contains "Congratulations" → Spam
- IF email is from "trusted@company.com" → Not Spam

### ✅ ML-Based Approach:

- Train an **ML classifier (e.g., Naïve Bayes, Random Forest, Neural Networks)** on **thousands of labeled emails**.
- Instead of predefined rules, the model **learns statistical relationships** between words, sender details, and past spam classifications.

# ML intuitions



# How ML Learns from Data & Adapts Incrementally

**Initial Learning:** The classifier learns **patterns in income & spend** to separate regular and risky customers.

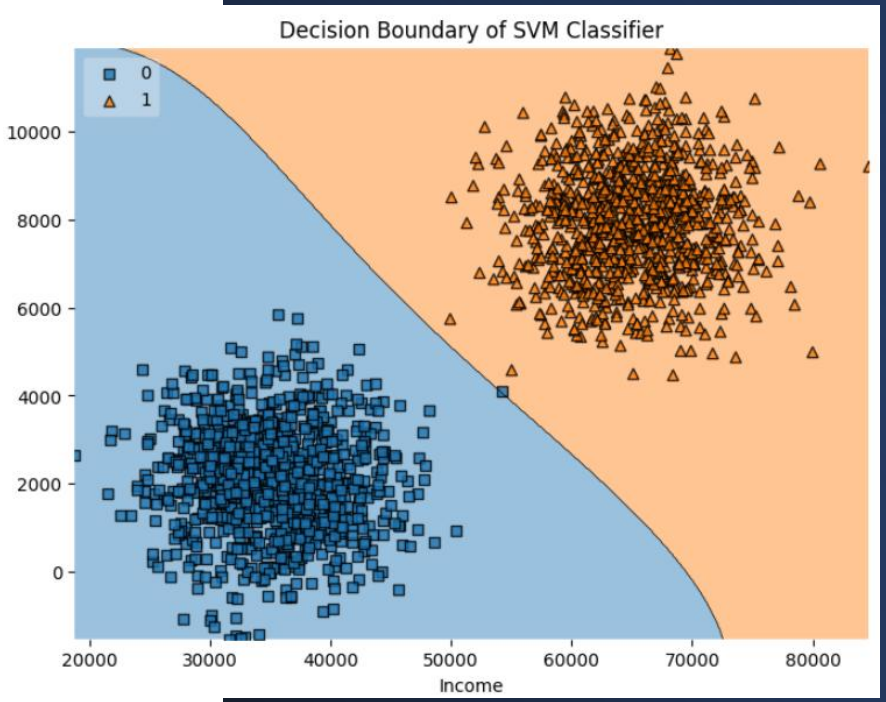
**Decision Boundary Formation:** The model creates a **boundary** between the two classes based on training data.

**Incremental Learning:** When new data is introduced, the model updates its **weights** using partial_fit(), refining its decision-making.

**Decision Boundary Adjustment:** The **boundary shifts** as the model learns from new data, improving its ability to generalize.

# Initial training and decision boundary

- The **RBF kernel** in the SVM helps create a non-linear decision boundary.
- The gamma=0.1 setting allows the model to capture **some curvature** in the boundary while avoiding excessive complexity.
- As a result, the decision boundary is **not a straight line** but adapts to the density of the data points.



# With Additional data

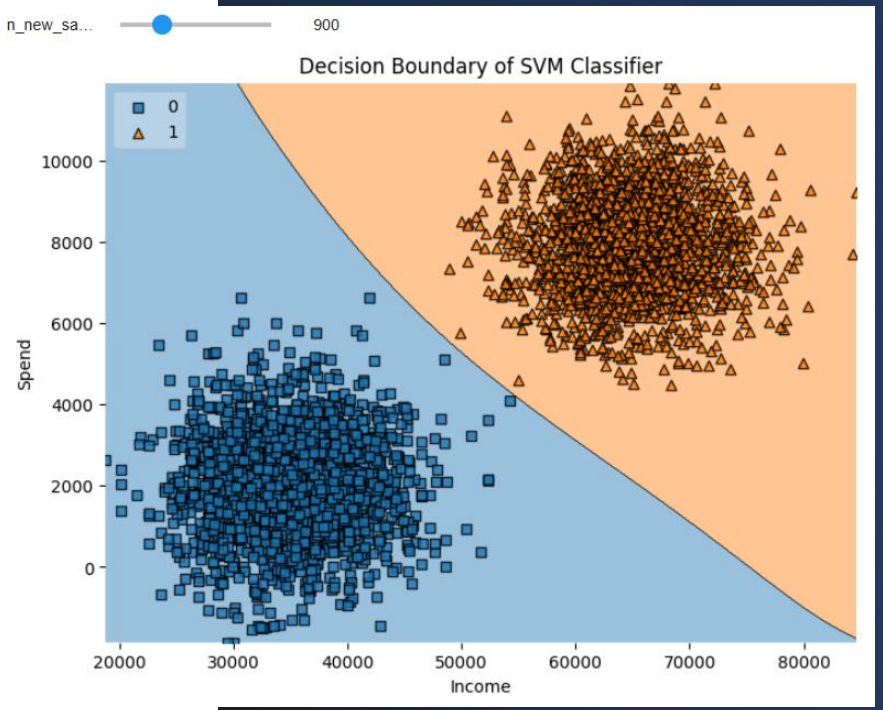
**If the new data follows the same distribution as the original dataset:**

•The boundary remains **mostly stable** with slight refinements based on additional support vectors.

**If the new data has different characteristics (e.g., more overlap between classes):**

•The boundary may become **less distinct** (smoother transition between classes).

•The decision region for one class may shrink or expand, depending on the new samples.



# Rule-Based AI → Clustering/ Anomaly Detection

**Example: Fraud Detection in Banking**

🔴 **Rule-Based Approach**:

- IF transaction > $10,000 & outside home country → Flag as Fraud.
- Problem: Many genuine users also **travel** and make large transactions.

✅ **ML-Based Approach**:

- Use **Anomaly Detection (e.g., Autoencoders, Isolation Forest)** to identify unusual spending patterns **without predefined rules**.

# Types of Agents

# Reflex Agents

**Concept :** Acts instantly on the current input.

- No memory, no reasoning about past or future.
- Great for quick, repetitive, rule-based actions.

## LangGraph Mapping

- **Nodes**:
  Input → Tool Call → Output
- No memory store, no intermediate state updates.

## Example : Currency Converter Bot

- User: "Convert 100 USD to EUR"
- Agent: Calls currency API → returns conversion rate.

# Reflex Agent

[User Message]

↓

[Decision Node: Parse currency & amount]

↓

[Call: Currency API Tool]

↓

[Send back converted value]

# ReAct Agents

**Definition**

- Follow a **Reason** → **Act** → **Observe** → **Repeat** cycle.
- LLM explicitly separates reasoning steps from actions.
- Uses observations from tools to guide next reasoning step.
- **LangGraph Support:** create_react_agent makes this quick to set up.

- **Example**:: Medical literature review agent:
- Reason: “Need to find latest EGFR lung cancer studies”
- Act: Call search_pubmed
- Observe: Found 15 papers
- Reason: “Now fetch abstracts”
- Act: Call fetch_abstracts
- Summarize.

[Reason Node] → [Tool Call Node] → [Observation Node]

# Tool-Calling Agents

**Definition**

•Core ability is **invoking tools** (APIs, databases, scripts).

•Often used when external actions or data retrieval are required.

**Example**:

Customer service agent that:

•Looks up order status from an internal database.

•Calls a shipping API to update tracking info.

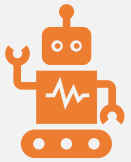
•Sends confirmation email.

**LangGraph Flow**

css

[User Request] → [Intent Detection] → [Tool Node(s)] → [Response Node]

# What is Agentic AI



Agentic AI is a subset of agentic systems, focusing on AI-driven agents with sophisticated cognitive abilities

often powered by modern AI models like transformers or RL algorithms.

It contrasts with earlier agentic systems (e.g., rule-based or reactive agents) by emphasizing intelligence, generalization, and human-like interaction.

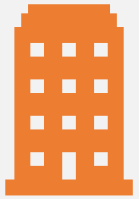
# Intelligence, Generalization, human-like interaction

**Intelligence**: The ability to reason, solve complex problems, and make context-aware decisions, often using advanced AI techniques like large language models (LLMs) or reinforcement learning (RL).

**Generalization**: The capacity to apply learned knowledge to new, unseen scenarios, unlike narrowly specialized systems.

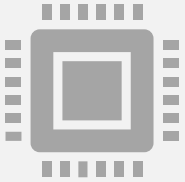
**Human-like Interaction**: The ability to engage with humans naturally, understanding nuanced intent and responding in a conversational or intuitive manner.

# contrast with



**Rule-Based Agents**:

Operate on predefined, deterministic rules (e.g., if-then logic in expert systems like MYCIN). They lack flexibility and cannot adapt beyond their programmed rules.



**Reactive Agents**: Respond to environmental stimuli in real-time using simple mappings (e.g., Rodney Brooks' subsumption architecture for robots). They are fast but lack reasoning or long-term planning.

# mean Agentic AI is inherently "probabilistic“?

In the context of AI and agentic systems, **probabilistic** refers to systems that:

- Use **probability distributions** to model uncertainty, make decisions, or predict outcomes.
- Incorporate **stochastic** (random) processes, as opposed to deterministic rules.
- Rely on statistical methods, such as Bayesian inference, probabilistic graphical models, or neural networks with probabilistic outputs (e.g., softmax in LLMs).

# Examples of probabilistic approaches in AI

- **Machine Learning Models**: Neural networks (e.g., LLMs) output probability distributions over possible actions or responses (e.g., next-word prediction in GPT models).
- **Reinforcement Learning**: Agents model uncertainty in environments using probabilistic policies (e.g., choosing actions based on a probability distribution over Q-values).
- **Bayesian Agents**: Use Bayesian reasoning to update beliefs based on new evidence, handling uncertainty explicitly.

# LLM-Centric Agent Architecture

**LLM**: Central brain for reasoning and communication.

**Memory**: Short-term (context window) and long-term (vector store).

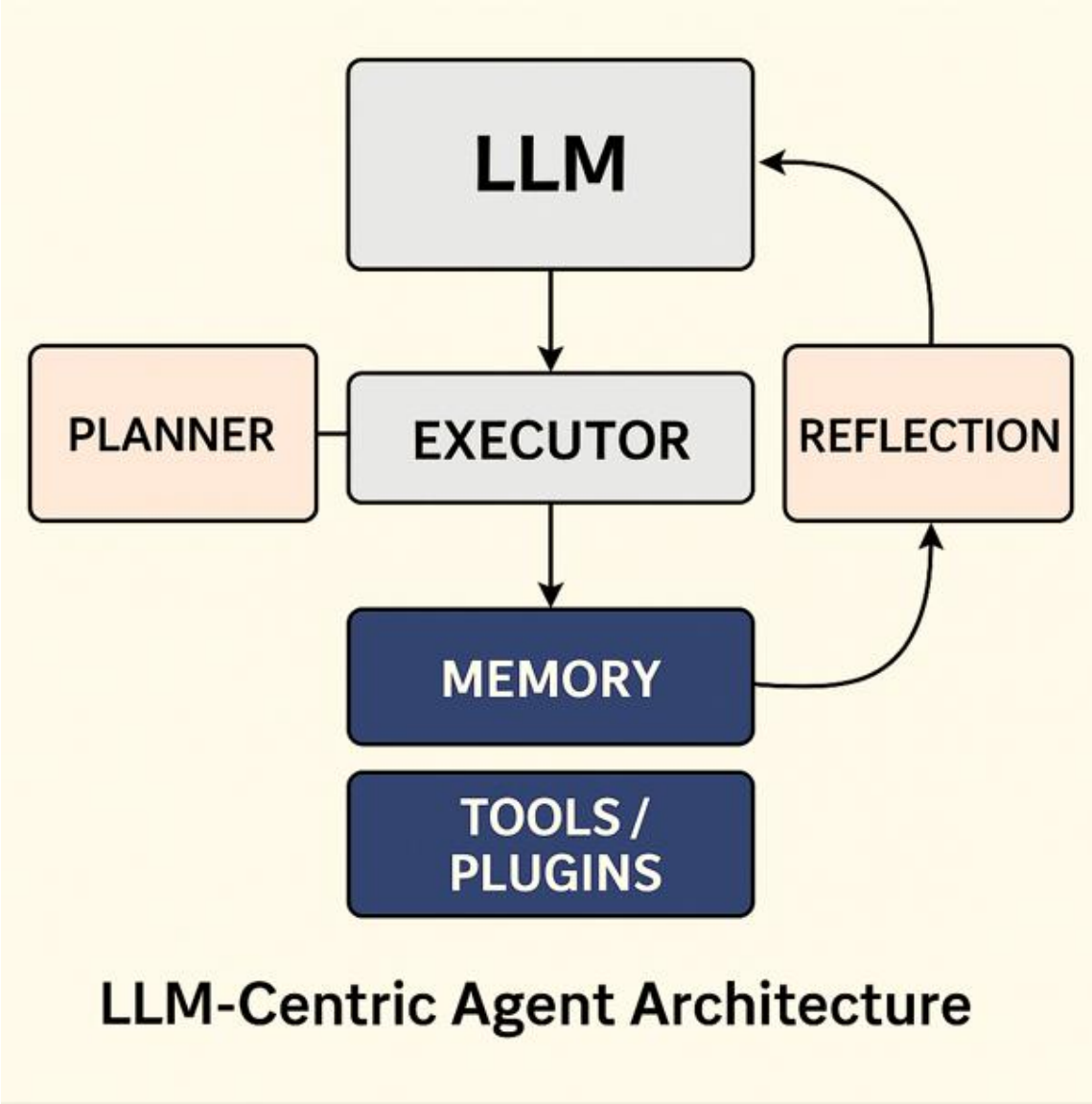
**Tools/Plugins**: APIs, databases, search, file systems.

**Planner**: (Optional) Generates multi-step plans/goals.

**Executor**: Executes steps, calls tools, gathers results.

**Reflection Loop**: Re-assesses outcomes and adjusts plans.

✅ Example: LangChain, OpenAI Function Agents, AutoGPT.



## Core Agent Capabilities

- Use of Agents
- Hand-off and Multi Agents
- Autonomy & Decision Making
- Learning & Adaptation

## Input & Perception

- Input Modalities
- Perception

## Processing & Orchestration

- Workflow Orchestration
- Memory/Vector DB
- Context Awareness / State Management
- Scalability & Parallelism

## Tools & External Integration

- Tools/APIs

## Output Generation & Formats

- Output Generation
- Multi-modal Output

## Reliability & Robustness

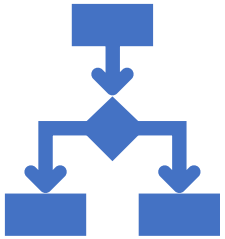
- Error Handling
- Fallbacks
- Retries

## Security, Ethics & Governance

- Security & Guardrails
- Ethical Considerations

## Observability & Extensibility

- Callbacks/Hooks
- Tracing/Logging



# Core Agent Capabilities

**Use of Agents**

**Hand-off and Multi Agents**

**Autonomy & Decision Making**

**Learning & Adaptation**

# Agents

**Definition:**

Deploy multiple intelligent agents, each with specialized skills, to handle **different tasks** or knowledge domains.

**Example: In an AI-powered hospital management system:**

**Patient Intake Agent**: Handles appointment scheduling and collects patient history.

**Diagnostics Agent**: Analyzes lab reports and imaging results for preliminary assessments.

**Insurance Claim Agent**: Prepares claim forms and validates documents against policy requirements. Each agent focuses on its specialty, reducing errors and speeding up hospital operations.

# Hand-off and Multi Agents

- **Definition:**
  - Seamlessly **transfer work between agents** based on the type of request or expertise needed, without user disruption.
- **Example:** In an **AI legal assistant**:
  - A **General Legal Advisor Agent** starts the conversation about a client's issue.
  - Upon detecting that the case involves **intellectual property law**, it hands off to the **IP Specialist Agent**, carrying forward all case details so the client doesn't repeat anything.
  - Later, the **Court Document Filing Agent** takes over to prepare and submit the necessary legal paperwork.

# Autonomy & Decision Making

- **Definition:**
  - Agents **independently** decide on the best action based on goals, rules, and changing circumstances.
- **Example:** In an **autonomous fleet management system**:
- Delivery drones monitor weather and traffic data in real time.
- If heavy rain is detected along the planned route, the drone autonomously decides to:
  - Delay the delivery, **or**
  - Switch to an alternative nearby drone with a rainproof payload system.
    No human intervention is required for the decision.

# Learning & Adaptation

- **Definition:**
  - Agents **evolve** and improve their performance over time using feedback and new data.

- **Example:** In an **AI tutoring platform**:
  - Initially, the Math Tutor Agent explains algebra problems using a single approach.
  - Over time, it analyzes which explanations lead to faster student understanding.
  - It learns to adapt — using visual diagrams for visual learners, or step-by-step breakdowns for analytical learners — improving accuracy and engagement.

# nothing here happens in a “self-created” vacuum

**Every agent starts with:**

**Rules** — written by humans (“If weather is bad, reroute delivery”).

**Abilities** — code or AI models trained to do a certain task (“Summarize a PDF”, “Book a ticket”).

**Triggers** — signals for when to act (“User asked for hotel → pass to hotel agent”).

**The *“hand-off,” “autonomy,” and “learning”* are just the result of:**

Good programming

Pre-trained AI models

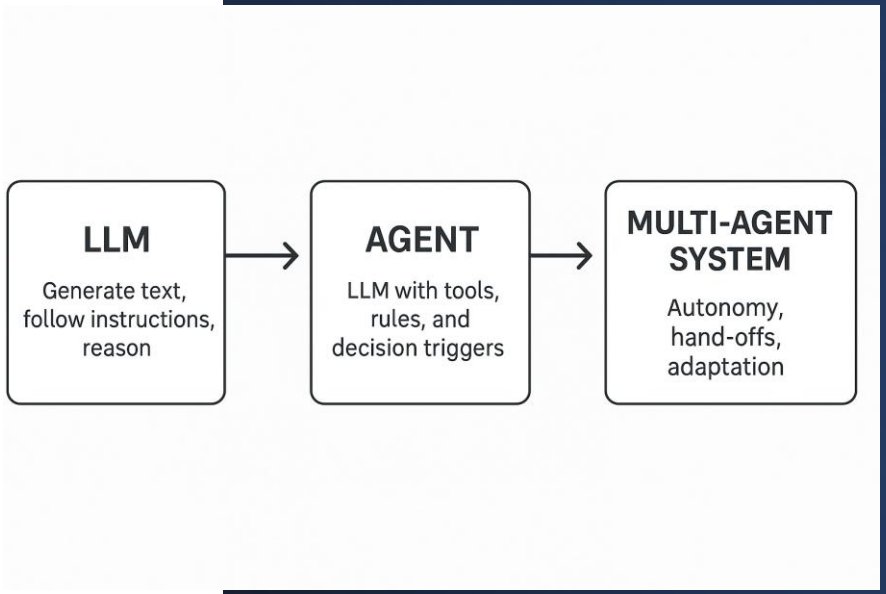
Optional feedback loops for improvement

# LM vs Agent Reality Check

- **LLM alone** = Great at generating text, following instructions, and reasoning — *but it's reactive*. It won't act unless you prompt it.

- **Agent built on LLM** = LLM + predefined **tools**, **rules**, **decision triggers**, and **memory**. This lets it *act* and *chain tasks*.

- **Autonomy, hand-offs, adaptation** = Come from **human-designed architectures** (LangGraph, AutoGen, custom Python orchestration), *not* from the LLM just "figuring it out."



# Input & Perception

**Input Modalities**
**Perception**

# Input Modalities

- *Meaning* :
  - The different forms of **data** an agent can receive.
- *Example:* A personal AI assistant might take
  - **Text**: typed questions (“What’s my schedule today?”)
  - **Voice**: spoken commands (“Set a timer for 5 minutes”)
  - **Images**: a photo of a receipt to extract totals
  - **Sensors**: temperature readings from a smart thermostat.

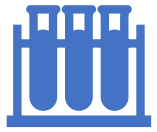
# Perception

- Meaning:
  - The agent's ability to interpret those inputs in a meaningful way.

- *Example:* The same assistant
  - Recognizes a photo as a receipt and extracts the purchase amount.

  - Understands "Set a timer for 5 minutes" as an action request, not casual conversation.

  - Detects from a sensor reading that the room is too cold and suggests turning on the heater.



# Medical Domain – Perception Examples

- **Lab Reports:**
  - AI reads “HbA1c: 8.2%” and understands it means “poorly controlled diabetes,” not just “a number.”
- **Medical Imaging:**
  - An AI sees a CT scan and identifies a small lung nodule, understanding it as a potential cancer risk.
- **Symptom Descriptions:**
  - Patient says, *“I feel chest tightness when I climb stairs.”*
  - AI interprets it as possible angina and suggests relevant follow-up tests.
- **Wearable Device Data:**
  - Continuous heart rate spikes detected → AI perceives this as possible arrhythmia needing review.

# Legal Domain – Perception Examples

- **Contract Review:**
  - AI reads a clause and recognizes it as a **non-compete agreement** that might conflict with local labor laws.
- **Case Law Search:**
  - AI finds a judgment and perceives it as a relevant precedent for “breach of contract” in the user’s jurisdiction.
- **Client Statements:**
  - Client says, *“My landlord changed the locks without notice.”*
  - AI interprets it as a possible case of “illegal eviction” under tenancy laws.
- **Document Classification:**
  - AI receives 100 scanned pages and identifies which are affidavits, which are evidence exhibits, and which are procedural forms.

# Processing & Orchestration



- Workflow Orchestration
- Memory/Vector DB
- Context Awareness / State Management
- Scalability & Parallelism

# Workflow Orchestration

- **Definition:**
  - Coordinating and sequencing tasks (or multiple agents' actions) so they work together smoothly and in the right order.

- **Core Functions**
  - **Task Decomposition:** Break large goals into manageable, specialized steps.
  - **Dynamic Routing:** Direct each sub-task to the most capable model, agent, or tool.
  - **Context Preservation:** Maintain shared state, history, and data across the workflow.
  - **Parallel & Sequential Execution:** Run tasks in the optimal order for speed and accuracy.
  - **Feedback Loops:** Use results from earlier steps to refine later ones.
  - **Error Recovery:** Detect failures and trigger fallback or retry mechanisms.

# Why LLM-Driven Orchestration Still Matters for Mature Workflows

- challenges:
  - Data scattered across structured systems
  - unstructured sources
  - Multiple alerts firing in parallel without context
  - Exceptions that require **reasoning** across silos, not just executing rules.

# Example - Healthcare (ICU)

- **Current reality**: ICU workflow already has tight EHR integrations and vitals-triggered alerts.
- **Pain point**: Clinicians get bombarded with dozens of alerts from vitals, labs, and medication logs — each system acting independently.
- **LLM orchestration**:
  - Pulls relevant vitals, lab results, and medication data from multiple hospital systems.
  - Merges them into a *single, coherent clinical narrative*.
  - Suggests *prioritized next steps* aligned with hospital protocols and the latest clinical guidelines.

# Memory / Vector DB

- **Definition:**
  - Storing important information so the agent remembers facts across conversations, and using a **vector database** to retrieve relevant chunks based on meaning (semantic search).
- **Examples:**
  - **Medical:**
    - Doctor's AI assistant recalls a patient's last 3 lab results from vector DB so it can compare trends when answering a new query.
  - **Legal:**
    - A case-prep bot retrieves the 5 most relevant past judgments from thousands of legal documents by semantic similarity, not just keyword match.

# Context Awareness / State Management

- **Definition:**
  - Keeping track of the conversation or process state so responses remain coherent and relevant.
- **Examples:**
  - **Medical:**
    - A telemedicine bot remembers that earlier in the chat the patient said they were pregnant — so it avoids recommending medications unsafe during pregnancy.
  - **Legal:**
    - During a multi-step deposition prep, the AI keeps track of which witnesses have already been covered and which are next.

# Scalability & Parallelism

- **Definition:**
  - Handling many tasks or conversations at once without slowing down.

- **Examples:**

  - **Medical:** Hospital AI triage system processing 500 simultaneous patient symptom submissions during an outbreak, each routed to different diagnosis agents.

  - **Legal:** An AI discovery assistant reviewing 10,000 documents in parallel to flag potential evidence within hours instead of weeks.

# Tools

are **the LLM’s hands and eyes**

# What do we mean by Tools / APIs in Agentic AI?

- A **function** (local or remote) that an agent can invoke to perform a task - it cannot or should not handle purely via text generation.
- It’s an action interface between the LLM’s reasoning and the outside world.
- Think of a Tool as : “A named, documented capability that the LLM can call, pass parameters to, and get structured results from.”

# Yes, there are tons of APIs already in existence

- weather APIs, financial data APIs, hospital EHR APIs —
- But …
  - They are **not directly LLM-usable** unless you wrap them in a Tool schema (with defined input/output).
  - it’s more about **creating callable capabilities** that the LLM can orchestrate

# Examples of Tools

**Data retrieval:** search_pubmed(query) → returns structured research abstracts.

**Business process:** generate_invoice(customer_id, items) → returns PDF or record ID.

**Specialized analytics:** predict_readmission(patient_record) → calls ML model for hospital readmission risk.

**Control actions:** restart_service(server_id) → restarts a cloud service instance.

# Emphasis on Creation of Tools

- This is where most AI learners miss the point
- You will almost always **need to create** custom tools.
- Why?
  - **Domain adaptation**: APIs return raw JSON; tools return clean, structured, and context-relevant results for the LLM.
  - **Orchestration** safety: Tools can enforce guardrails, logging, and pre/post-processing.
  - **Workflow alignment**: Tools are designed to fit into your exact process, not the generic API responses.

# Output Generation & Formats



agent’s ability to produce responses

Ability to generate rich responses that combine text, audio, images, video, or interactive elements.

# Output Generation

- **Definition:** The agent's ability to **produce responses** in the format best suited for the user's needs.

- Goes beyond "answering" — the response may be **narrative, structured, or executable**.

- Can adapt based on context:
  - Casual conversation → plain text.
  - Data analysis → tables, charts, code.
  - Accessibility → speech, captions.

# Examples

A finance agent outputs

- Summary text for a report.
- An Excel file for analysis.
- A PDF for printing.

A health chatbot gives

- Doctor-friendly clinical summary.
- Patient-friendly plain language.
- JSON for EHR integration.

# A design principle for Agentic AI systems

- it’s not just about **reasoning** and **deciding** what to do —
- it’s also about **delivering the result** in the most actionable and consumable form for the user or downstream system.
- good reasoning with bad output packaging is as useless as bad reasoning — the user can’t act on it.

# Multi-modal Output

- **Definition:** Ability to generate **rich responses** that combine **text, audio, images, video, or interactive elements**.
- Enables more engaging and effective communication.
- Critical for domains like **education, design, diagnostics, and simulations**.

# Examples

- **Education Agent**: Generates text explanations + diagram + audio narration.
- **Design Agent**: Outputs a design mockup + downloadable assets.
- **Medical Imaging Agent**: Sends text interpretation + annotated X-ray image.

# Seriously ?

Despite all the marketing noise, today's LLMs (even the big frontier ones) are **not truly** "click-and-get" process diagram or production-grade UI generators.

# What LLMs can actually do today

- **Suggest diagram structures in text form** (e.g., "three boxes, arrows from left to right, label them…").
- **Generate code** for diagramming tools like Mermaid, PlantUML, or **Graphviz**, which you then render.
- **Produce placeholder wireframes** using HTML/CSS/Tailwind or Figma plugin scripts — but they won’t be polished without human intervention.
- **Describe imagery** in detail for downstream image models (like DALL·E, Midjourney, Stable Diffusion).

# Reliability & Robustness

Error Handling

Fallbacks

Retries

# Error Handling

- *Detect, log, and respond to errors in a controlled manner without crashing workflows.*

- **Single-Agent Example:**
  - **medical report summarizer agent** encounters malformed text due to a PDF extraction bug. Instead of failing, it logs the error, skips the faulty section, and processes the remaining pages.

- **Multi-Agent Example:**
  - **healthcare RAG pipeline**, the *Document Loader Agent* throws an error when an API is down. The *Coordinator Agent* catches the exception, notifies the *Monitoring Agent*, and continues processing available documents.

# Fallback Strategies

- *Switch to alternative methods, models, or agents if the primary path fails.*

- **Single-Agent Example:**
  - A **translation agent** first uses GPT-5 for English-to-Hindi conversion. If it fails or times out, it switches to a fine-tuned local MarianMT model.

- **Multi-Agent Example:**
  - In a **medical diagnosis assistant**, if the *Image Analysis Agent* (NVIDIA DL model) is unavailable, the *Coordinator Agent* falls back to a *Text-Only Reasoning Agent* using existing patient notes.

# Retries & Recovery

- *Automatically reattempt failed operations, with safeguards to prevent infinite loops.*

- **Single-Agent Example:** A **web-search agent** retries fetching search results up to 3 times when the connection drops, adding a short delay between attempts.

- **Multi-Agent Example:** In a **financial data aggregation system**, if the *Stock Price Fetcher Agent* fails due to API throttling, the *Coordinator Agent* retries after 60 seconds and informs the *Alert Agent* to notify the user.

# Observability & Extensibility

- Let developers, operators, or automated monitors inspect
- Allow developers to extend the agent's behavior

# Tracing / Logging

- Capturing **what happened, when, and why** inside the agent—both for debugging and for compliance/audit purposes.
- **Example in Agentic AI:** Every step of a **multi-agent medical workflow** is logged:
  - Which agent handled the step (e.g., “Drug Interaction Checker”).
  - Inputs, outputs, and confidence scores.
  - Time taken for each call.
- **Benefits:**
  - Debugging failed reasoning chains.
  - Auditing medical decisions for legal compliance.
  - Identifying performance bottlenecks in multi-agent pipelines.

# Callbacks / Hooks

| | |
|---|---|
| **Definition** | Points in the agent's lifecycle where you can "hook in" extra logic—such as sending a notification, modifying data, or triggering another workflow—without touching the core pipeline. |
| **Example in Agentic AI** | **Event:** Agent finishes fetching patient lab results.<br>**Callback Action:** Automatically trigger a **data validation script** to check for anomalies before sending results to the diagnosis module. |
| **Sample Hook Points** | Before tool execution → validate input<br>After LLM reasoning → apply content filters<br>On error → send alert to monitoring dashboard |

# Security, Ethics & Governance

Security & Guardrails

Ethical Considerations

# Access Controls

- **How:**
  - Authenticate users (login, tokens, API keys).
  - Authorize actions using **role-based access control (RBAC)** or **attribute-based access control (ABAC)**.
  - Inside the agent's tool execution layer, check **user role** before allowing an action.

- **Example in Agentic AI:**
  - An agent has a query_patient_records() tool.
  - Code checks:

    ```
    if user.role != "doctor": raise PermissionError("You are not allowed to access patient records.")
    ```

  - Result:
    - Doctor: ✅ Query allowed.
    - Receptionist: ❌ Denied access.

# Content Filtering

**How:**

- Use LLM moderation APIs (e.g., OpenAI's Moderation API).
- Apply regex, keyword lists, or ML classifiers before showing LLM outputs to the user.
- Add **post-processing filters** to the agent's responses.

**Example:**

- Agent answers medical questions.
- A patient asks for **suicide methods** → the moderation layer blocks it and responds: "I cannot provide that information. If you are feeling distressed, please contact XYZ helpline."

# Why Agentic AI Is the Next Step

# From Passive to Proactive: Autonomous Planning and Execution

- **Reason:** Traditional AI systems often require explicit prompts and human intervention to perform tasks.
- In contrast, Agentic AI systems autonomously plan, **reason**, **act**, and adapt to achieve predefined goals.

- **Example:** In the finance sector, Agentic AI can autonomously reconcile transactions, detect anomalies, and generate reports without human oversight, streamlining financial operations.

# Complex Decision-Making: Handling Multi-Step, Interdependent Tasks

- **Reason:** Agentic AI can manage **intricate workflows** that involve multiple steps and dependencies, making decisions based on **real-time data and evolving conditions**.

- **Example:** In healthcare, Agentic AI systems can coordinate patient care by scheduling appointments, sending reminders, and **adjusting treatment plans** based on patient responses and test results.

# Scalability & Efficiency: Automating Complex Workflows

- **Reason:** Agentic AI enables the **automation** of complex business processes that span multiple teams, tools, and decision layers, resulting in significant efficiency gains.

- **Example:** In retail, Agentic AI can manage inventory levels, forecast demand, and reorder stock autonomously, ensuring optimal stock levels and reducing manual intervention.

# Human-Like Interaction: Proactively Gathering Information and Collaborating

- **Reason:** Agentic AI systems can engage in dynamic interactions, asking clarifying questions and collaborating with humans to achieve objectives, mimicking human problem-solving behaviors.

- **Example:** In customer service, Agentic AI can handle initial customer inquiries, escalate complex issues to human agents, and follow up to ensure resolution, providing a seamless customer experience.

# When to Switch to Agentic AI — Probable Real-World Scenarios

| Scenario | Why Agentic AI Fits |
| --- | --- |
| **Customer Support Automation** | Handles complex queries, escalates intelligently, integrates knowledge bases and CRM. |
| **Medical Diagnosis Assistance** | Combines symptoms, lab results, guidelines, and recommends personalized treatment plans. |
| **Financial Portfolio Management** | Monitors markets, rebalances portfolios, and alerts on risks with multi-step decision logic. |
| **Content Creation & Moderation** | Plans content series, edits drafts, flags policy violations automatically. |
| **Research Assistance & Data Synthesis** | Searches literature, extracts key points, summarizes, and generates reports autonomously. |

# Agentic AI is ideal when tasks are:

Complex, multi-step, and dynamic

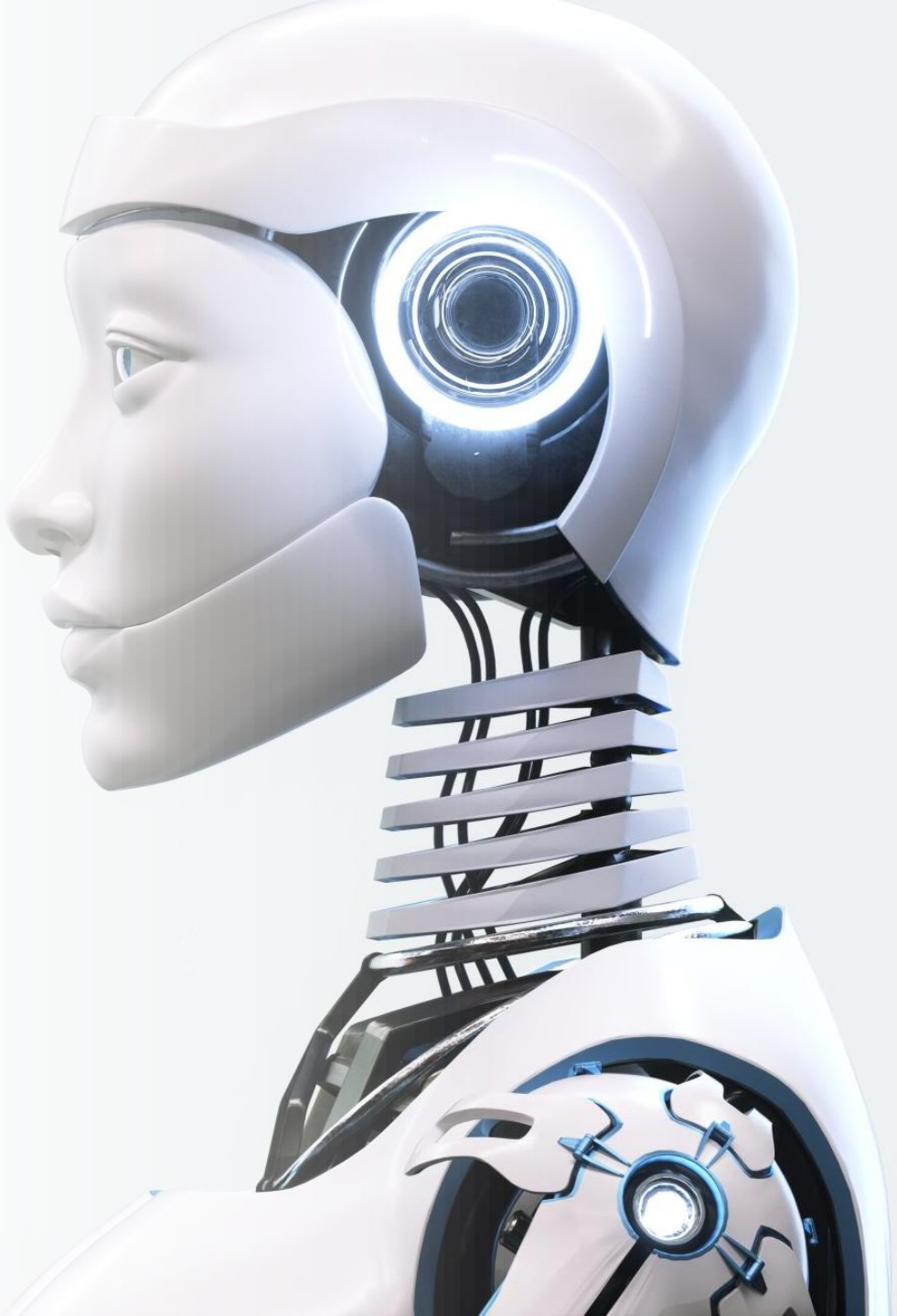
Require interaction with multiple data/tools

Benefit from autonomous reasoning and planning

Need human oversight but less manual intervention

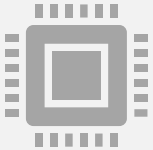
# Enabling transformation Using AI

AI **Mental model**



# The AI Awareness Paradox: High Awareness vs Low effective implementation.

Many enterprises are aware of AI's potential and express interest



but a significant gap exists between this awareness and the actual successful deployment and integration

Less tangible business value at scale.

# Why

Insufficient data infrastructure, quality, and accessibility.

Shortage of skilled AI talent (data scientists, ML engineers, AI ethicists).

Organizational silos hindering cross-functional AI initiatives.

Fear of the unknown and resistance to adopting new technologies.

Unrealistic expectations about the speed and ease of AI implementation.

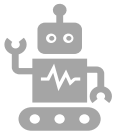
# Analyze the capability implementation gaps

## Knowledge Gap:

Business leaders lack a deep understanding of what AI can realistically achieve and how it can solve their specific problems. *Example: A marketing team is aware of AI for personalization but doesn't understand how to define the right customer segments or choose the appropriate AI algorithms.*

## Prioritization Gap:

Organizations struggle to identify and prioritize AI projects that offer the highest potential business value and align with strategic goals. *Example: A company invests in a flashy but low-impact AI project while neglecting a high-ROI opportunity in supply chain optimization.*



## Implementation Gap:

Challenges in translating AI concepts and pilot projects into robust, production-ready systems. *Example: A successful AI model developed by the data science team fails to integrate with the existing IT infrastructure or business workflows.*

## Scaling Gap:

Difficulty in expanding successful AI deployments across different business units or processes. *Example: An AI-powered customer service chatbot works well for one product line but struggles to handle the complexity of other offerings.*

# Recognize the mental models for AI contextualization

Understanding different mental models helps in framing and applying AI effectively to specific business contexts.

These models provide different lenses through which to view AI's capabilities and potential.

# Examples

robotic process automation with AI - reduces human effort by handling routine, structured, or rule-based tasks.

**Customer Support**: AI chatbots handle Tier-1 support queries.

**Invoice Processing**: Intelligent document processing to extract data from PDFs/emails.

**Recruitment**: Screening resumes using AI-powered parsing and filtering.

**Manufacturing**: AI-powered robotic arms on production lines using computer vision.

# AI as Augmentation

| Healthcare | Marketing | Legal | Finance |
| --- | --- | --- | --- |
| • Radiologists using AI to highlight suspicious regions in MRI scans. | • AI-assisted campaign optimizations based on user behavior. | • AI tools summarizing case law or extracting clauses from contracts. | • Portfolio managers getting anomaly alerts for high-risk transactions. |

# AI as Innovation

| | |
|---|---|
| **Media** | Generative AI used to create synthetic actors, music, or video content. |
| **Fashion/Retail** | AI-driven virtual try-ons using computer vision and avatars. |
| **Education** | Personalized learning platforms adapting to student styles via LLMs. |
| **Healthcare** | Personalized Health Plans. |

# AI as Prediction

| | |
|---|---|
| **Retail** | Forecasting demand for inventory and product restocking. |
| **Energy** | Predicting energy consumption in smart grids. |
| **Insurance** | Risk scoring for policy underwriting. |
| **Transport** | Predictive maintenance for fleet management using sensor data. |

# AI as Interaction

AI transforms how humans interface with digital systems or devices.

- **Voice Assistants**: Siri, Alexa, and Google Assistant enabling voice commands.
- **Customer Engagement**: Conversational AI for 24x7 query resolution.
- **Accessibility**: Real-time sign language interpretation using AI vision models.
- **Immersive Tech**: AI avatars and NPCs in gaming/VR environments that interact dynamically.

# From Rule-Based to Pattern Recognition

🔄 Old: “If X, then Y” logic

⚡ New: “If it *looks like* X, it’s *probably* Y”

| Mental Model | How it addresses this shift | Example |
|---|---|---|
| AI as Automation | Replaces static business rules with models trained on patterns | Chatbots recognize intent from phrasing, not just keywords |
| AI as Augmentation | Finds insights hidden in messy or complex data | AI in diagnostics spots patterns in radiology images |
| AI as Innovation | Uses learned data patterns to create new experiences | Spotify recommends songs based on listening behavior |
| AI as Prediction | Forecasts based on data trends, not hard-coded logic | Predicting customer churn from subtle behavioral shifts |
| AI as Interaction | Understands user context beyond command patterns | Voice assistants recognize intent across accents & styles |

# From Deterministic to Probabilistic Thinking

🔄 Old: Always expect a “yes or no”

⚡ New: Think in probabilities (“90% chance this user will churn”)

| Mental Model | How it adapts | Example |
|---|---|---|
| AI as Automation | Flags anomalies or fraud with confidence levels | “This transaction has a 94% chance of being fraudulent” |
| AI as Augmentation | Supports humans with ranked suggestions, not absolute answers | AI suggests 3 likely diagnoses, doctor makes the final call |
| AI as Innovation | Creates content or responses that vary probabilistically | LLMs generate multiple ad variants or narratives |
| AI as Prediction | Naturally relies on uncertainty modeling | Predicting delivery delays based on probabilistic models |
| AI as Interaction | Handles ambiguous queries by ranking likely intents | “Book a table” → shows top 3 restaurant matches |

# From Fixed Workflows to Adaptive Processes

🔄 Old: Pre-set rules and steps

⚡ New: Flows adapt to user data, context, and goals

| Mental Model | How it supports adaptivity | Example |
|---|---|---|
| AI as Automation | Automates approvals or tasks based on changing criteria | Dynamic loan approval workflows |
| AI as Augmentation | Recommends next steps based on what's happening | Sales assistants suggest next-best-actions |
| AI as Innovation | Enables workflows that personalize per user or event | Personalized learning paths in EdTech |
| AI as Prediction | Adjusts plans based on real-time trends | Dynamic pricing models in e-commerce |
| AI as Interaction | Adapts responses or UI flows based on tone, sentiment, or behavior | Chatbots escalate if frustration is detected |

# From Expertise Scarcity to Expertise Augmentation

🔄 Old: Only domain experts knew what to do

⚡ New: AI helps *non-experts* make better decisions

| Mental Model | How it democratizes expertise | Example |
|---|---|---|
| AI as Automation | Handles routine expert tasks automatically | AI reviews legal contracts for compliance |
| AI as Augmentation | Empowers professionals with insights formerly buried in data | Doctors use AI to catch rare diseases in scans |
| AI as Innovation | Makes expert-level tools available to consumers | DIY tax software with AI-guided help |
| AI as Prediction | Surfaces patterns that only experts could spot before | Predictive tools for stock traders or small biz owners |
| AI as Interaction | Offers intuitive interfaces to access complex capabilities | AI tutors helping kids solve math step-by-step |

# Thanks &

# Happy Learning !!